

- EXAMEN DE 1^{ère} SESSION -

NOM (en majuscules) :	Date : 17/04/2017
Prénom :	Section / Année :
Local :	

UE : Principes d'algorithmique et programmation – C#

Epreuve : Examen final

Professeur : François Koeune

-
- Matériel autorisé : ordinateur fixe de l'EPHEC (sans accès internet), Visual Studio et accès à la documentation MSDN.
 - Durée : 3 heures.
-

Ne pas oublier :

- Lisez attentivement les questions avant de répondre.
- Toutes les réponses doivent être stockées sous forme de projets (dont les noms sont précisés ci-dessous) sur votre lecteur G:, où je pourrai les récupérer.
- Si vous le désirez, vous pouvez me demander de vérifier que vos réponses sont bien stockées au bon endroit, à condition de le faire avant la fin de l'examen.
- Si vous avez des difficultés pour accéder à la documentation MSDN, effectuez l'opération décrite en dernière page.

Première partie, sans utiliser d'ordinateur

Question 1.

Ecrivez **exactement** ce qui s'affichera à la console si l'on exécute le programme suivant :

```
static void Main(string[] args)
{
    int i = 1;
    while (i < 5000)
    {
        Console.WriteLine(i);
        i = i * 10;
    }
}
```

Remarques:

- Ce programme ne contient pas d'erreurs
- Inutile de rédiger un texte expliquant ce que le programme fait, indiquez juste le résultat affiché.

Réponse :

Question 2.

Le programme ci-dessous est censé tester si un nombre est divisible par 5.

```
static bool DivisiblePar5(int Nombre)
{
    bool Divisible;
    int test = Nombre % 5;
    if (test == 0) Divisible = true;
    else Divisible = false;
}

static void Main(string[] args)
{
    bool Res = DivisiblePar5(14);
    if (Res == true)
        Console.WriteLine("Nombre divisible par 5");
    else
        Console.WriteLine("Nombre non divisible par 5");
}
```

Ce programme comporte cependant une erreur. Corrigez-la en indiquant lisiblement votre correction dans le code ci-dessus (il n'y a qu'une correction à apporter).

Question 3.

Expliquez ce que fait la fonction `f()` ci-dessous, qui ne contient pas d'erreur :

```
static double f(double a, double b)
{
    if (b != 0)
        return a / b;
    else return -1;
}
```

Remarque : vous ne devez pas décrire ligne par ligne ce que fait le code, mais décrire l'effet global de la fonction (par exemple « cette fonction affiche les nombres de 1 à 10 »).

Réponse :

Deuxième partie, sur PC

L'objectif des questions 4 à 12 est de programmer une partie d'une application de mots croisés. Chacune des questions ci-dessous vise à écrire un élément contribuant à la solution. La dernière question vise à démontrer leur utilisation. Vous pouvez évidemment utiliser pour une réponse les fonctions écrites pour les réponses précédentes.

Si vous ne pouvez pas répondre à une des questions, vous pouvez néanmoins indiquer la signature qu'aurait la fonction correspondante, par exemple

```
int Tartempion(string Nom, char Initiale)
```

et utiliser cette fonction pour la suite de l'examen : ceci permettra de valider vos autres réponses.

Principe du jeu des mots croisés

Le jeu utilise une grille de 10 cases sur 10, dont quelques-unes sont noircies pour indiquer qu'elles ne peuvent pas être remplies. Le but du jeu est d'écrire une lettre dans chacune des cases non-noircies afin de former des mots qui se croisent. Le début et la fin de chaque mot correspondent, soit à des cases noircies, soit aux bords de la grille.

Par exemple, sur le morceau de grille ci-contre, on ne peut écrire un mot en 4^{ème} ligne que si sa 2^{ème} lettre est un U et sa 4^{ème} est un T. Ce mot doit en outre compter 6 lettres.

			E		T		
			G		A		
			O	T	I	T	E
			U		T		
			T				

Des mots croisés comportent normalement une série de définitions, mais nous ne prendrons pas ceci en compte pour cet examen : notre but est simplement d'écrire des fonctions permettant de remplir la grille avec des mots-candidats.

Implémentation du jeu

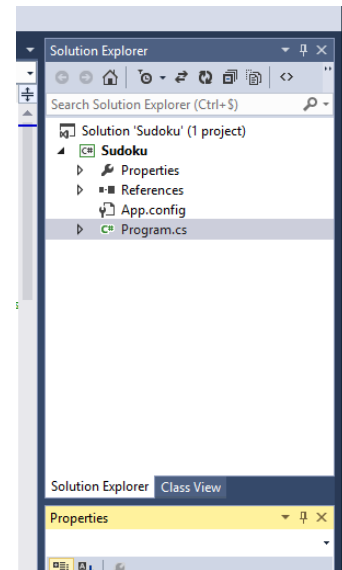
Nous représenterons une grille de mots croisés par un tableau de 10x10 éléments de type char. La première coordonnée indique le numéro de la ligne, la deuxième le numéro de la colonne : par exemple, l'élément Grille[5,2] correspond à la case située en 6^{ème} ligne et 3^{ème} colonne de la grille (puisque les indices commencent à 0). La valeur stockée à cet endroit est :

- Soit le symbole '*' pour indiquer une case noircie.
- Soit une lettre de l'alphabet. Seules les 26 lettres majuscules sont prises en compte (un 'é' est par exemple noté 'E').
- Soit un point '.' pour indiquer que la case n'est pas encore remplie.

Chaque position de départ pour un mot sera indiquée par 3 coordonnées : un numéro de ligne, un numéro de colonne et une lettre, 'h' ou 'v', pour indiquer si le mot doit être écrit horizontalement ou verticalement. Par exemple, le mot « EGOUT » ci-dessus se trouve aux coordonnées 0, 3, 'v'.

Projet de départ

- Récupérez le projet “MotsCroises”, disponible dans le répertoire “K:\IN1-Principes algo. et programm” et copiez-le sur votre lecteur “G:”. Ce projet vous servira de base de travail. C’est également là que vous écrirez vos réponses aux questions ci-dessous.
 - *Remarque : si le code n’apparaît pas lorsque vous ouvrez le projet avec Visual Studio, cliquez sur “Program.cs” dans votre explorateur de solutions (cf. figure ci-contre).*
- Ce projet comporte déjà une fonction qui génère et renvoie une grille pré-calculée (correspondant à la figure ci-dessus) permettant de tester votre programme. Il vous suffit d’appeler cette fonction au début de votre Main().



Questions (suite)

Question 4.

Ecrivez une fonction `AfficherGrille()` qui reçoit un tableau de 10x10 char correspondant à une grille (dont certaines cases peuvent être déjà remplies) et affiche cette grille à la console. Vous pouvez supposer (ici et pour toutes les questions qui suivent) que le tableau reçu est bien formé (pas besoin de vérifier si la taille et le contenu sont corrects).

Vous pouvez afficher les cases noircies avec le caractère “*”.

Question 5.

Ecrivez une fonction `CoordonneesValides()` qui reçoit 3 paramètres (un numéro de ligne, un numéro de colonne, une lettre indiquant le sens horizontal ou vertical) et renvoie une information indiquant si ces coordonnées sont valides. Des coordonnées sont valides si les numéros de ligne et de colonne, ainsi que le sens, sont des valeurs autorisées (il n’est pas demandé de vérifier ce qui se trouve à cet endroit de la grille, par exemple une case noircie).

Question 6.

Ecrivez une fonction `LireCoordonnees()` qui demande à l’utilisateur d’entrer au clavier 3 coordonnées :

- un numéro de ligne,
- un numéro de colonne,
- une lettre ‘h’ ou ‘v’ indiquant le sens horizontal ou vertical.

Si les éléments entrés sont valides, la fonction renvoie ces éléments sous la forme de deux entiers et un char, transmis en paramètres out. Sinon, la fonction demande à nouveau à l’utilisateur d’entrer 3 coordonnées, et ainsi de suite jusqu’à obtenir des valeurs valides.

Pour simplifier :

- Il n’est pas nécessaire d’indiquer à l’utilisateur quels éléments sont invalides : si un élément est incorrect, il doit tout réentrer.
- Vous pouvez supposer que l’utilisateur entrera des données du bon type : il entrera par exemple bien un entier (mais pas forcément entre 0 et 9) comme numéro de ligne.

Question 7.

Ecrivez une fonction `LettrePossible()` qui :

- Reçoit 4 paramètres : une grille, un numéro de ligne, un numéro de colonne et un char contenant une lettre majuscule.
- Renvoie une information indiquant si la lettre reçue est autorisée à cet endroit. La lettre est autorisée si la position est bien à l'intérieur de la grille et qu'une de ces conditions est vérifiée:
 - La position correspondante dans la grille n'est pas encore remplie.
 - La position contient déjà une lettre, mais identique à celle passée en paramètre.

Remarques :

- La fonction teste si l'opération est possible, mais ne modifie pas la grille.
- Pour simplifier, vous pouvez supposer que la lettre reçue est bien une majuscule entre A et Z.

Question 8.

Ecrivez une fonction `EcrireLettre()` qui :

- Reçoit 4 paramètres : une grille, un numéro de ligne, un numéro de colonne et un char contenant une lettre majuscule.
- Si l'opération est autorisée, écrit la lettre reçue à cet endroit dans la grille. Sinon, rien n'est modifié.
- Renvoie une information indiquant si la lettre a été écrite.

Pour simplifier, vous pouvez supposer que la lettre reçue est bien une majuscule entre A et Z.

Question 9.

Ecrivez une fonction `MotPossible()` qui :

- Reçoit 5 paramètres : une grille, des coordonnées (un numéro de ligne, un numéro de colonne et un char indiquant une direction) et une chaîne de caractères correspondant à un mot à écrire dans la grille.
- Renvoie une information indiquant si le mot reçu peut être écrit à cet endroit, c'est-à-dire si chacune des lettres du mot peut être écrite à l'emplacement où elle devrait être placée dans la grille (position encore libre, ou contenant la même lettre).

Remarques :

- Vous pouvez supposer que le mot reçu a le bon format (lettres majuscules).
- La fonction teste si l'opération est possible, mais ne modifie pas la grille.
- Il ne vous est pas demandé de vérifier si le mot est trop court.

Par exemple, dans la grille de la figure ci-dessus :

- « NUITEE » est un mot possible en (3,2, 'h'), car les lettres déjà présentes correspondent.
- « NUIT » est également possible, même s'il n'occupe pas tout l'espace (mais « NUITEES » ne le serait pas, car il est trop long).
- « EXAMEN » n'est pas possible en (3,2, 'h').
- « RAT » n'est pas possible en (2,2, 'v'), car la première case est noircie.

Question 10.

Ecrivez une fonction `EcrireMot()` qui :

- Reçoit 5 paramètres : une grille, des coordonnées (un numéro de ligne, un numéro de colonne et un char indiquant une direction) et une chaîne de caractères correspondant à un mot à écrire dans la grille.
- Si l'opération est autorisée, écrit le mot reçu à cet endroit dans la grille. Sinon, rien n'est modifié.
- Renvoie une information indiquant si le mot a été écrit.

Vous pouvez supposer que le mot reçu a le bon format (lettres majuscules).

Question 11.

Ecrivez une fonction `ListeMotsPossibles()` qui :

- Reçoit 5 paramètres : une grille, des coordonnées (un numéro de ligne, un numéro de colonne et un char indiquant une direction) et une liste de chaînes de caractères correspondant à des mots à tester.
- Renvoie, *sans modifier la liste initiale*, la sous-liste des mots qui pourraient être écrits à cet endroit.

Par exemple, avec la grille de la figure, si la fonction reçoit les coordonnées (3, 2, 'h') et la liste < « CHEVAL », « NUIT », « TRAVAILLEUR », « NUITEE » >, elle doit renvoyer < « NUIT », « NUITEE » >. Notez que la fonction reçoit plusieurs mots, mais une seule position.

Vous pouvez supposer que les mots reçus ont le bon format (lettres majuscules).

Question 12.

Démontrez le fonctionnement de votre programme en écrivant dans `Main()` le code mettant en œuvre le comportement suivant :

1. Charger la grille de test.
2. Vérifier si l'on peut écrire un 'A' en position (1, 5) : la réponse est oui.
3. Vérifier si l'on peut écrire un 'E' en position (0, 5) : la réponse est non.
4. Vérifier si l'on peut écrire un 'E' en position (11, 35) : la réponse est non.
5. Ecrire un 'A' en position (0, 0).
6. Demander à l'utilisateur d'entrer 3 coordonnées, en vérifiant qu'elles sont correctes. Entrez (0, 3, 'v') pour effectuer l'opération suivante.
7. Vérifier si l'on peut écrire le mot « EGOUT » en (0, 3, 'v') : la réponse est oui.
8. Ecrire effectivement ce mot à cet endroit.
9. Tenter d'écrire le mot « DISTRIBUTEUR » en (0, 5, 'v') : cela ne fonctionne pas.
10. Tester quels mots de la liste < « CHEVAL », « NUIT », « TRAVAILLEUR », « NUITEE » > peuvent être écrits en (3, 2, 'h').
11. Afficher la grille.

L'objectif de la dernière question, qui n'est pas liée au jeu de mots croisés ci-dessus, est de montrer votre capacité à utiliser les outils de débogage (*debugging*)

Question 13.

Le projet TexteEnMaj, que vous trouverez dans le répertoire "K:\IN1-Principes algo. et programm", est censé lire un texte au clavier et le convertir en majuscules. Plus précisément, les lettres minuscules non accentuées sont converties en majuscules, les autres caractères sont reproduits tels quels. Par exemple, si le texte entré est « bonjour à tous, vous ALLEZ BIEN ? », le résultat doit être « BONJOUR à TOUS, VOUS ALLEZ BIEN ? ».

Ce programme comporte cependant plusieurs erreurs. Corrigez-le en vous aidant des outils de debugging.

En cas de problème pour accéder à la documentation

Si vous rencontrez des difficultés pour accéder à la documentation MSDN, vérifiez que l'option des préférences pour l'aide est bien sur « lancer dans la visionneuse d'aide » et pas sur « lancer dans le navigateur ».

